Faso Cikan

La messagerie souveraine de l'administration

Guide technique

Installation, exploitation et mise en production

DANS CE GUIDE

- Les commandes à taper, étape par étape (avec ou sans WSL)
- Ce que fait chaque service au démarrage
- Configuration de Synapse, Keycloak, Element et Caddy
- Sauvegarde, restauration et mises à jour
- Passage en production et dépannage

Hackathon 226 · Équipe Ladies coders, UJKZ

Version du 30/10/2026

Synapse 1.163.2 · Element Web 1.12.32 · Keycloak 26.7.5 · PostgreSQL 16 · Caddy 2.10

SOMMAIRE

Table des matières

0. Les commandes, dans l'ordre . . . 3

Étape 1 : Préparer Windows (une seule fois, sinon passez à l'étape 2) . . . 3

Étape 2 : Ouvrir le terminal Ubuntu et vérifier les outils . . . 3

Étape 3 : Copier le projet dans Linux . . . 3

Étape 4 : Générer la configuration . . . 4

Étape 5 : Construire et lancer (environ 10 minutes la première fois) . . . 4

Étape 6 : Faire approuver le certificat local . . . 4

Étape 7 : Vérifier . . . 4

Étape 8 : Se connecter . . . 4

Étape 9 : Tester la révocation . . . 5

Étape 10 : Arrêter, relancer, repartir de zéro . . . 5

0 bis. Sans le script : Docker Compose directement . . . 6

Étape 1 : Se placer dans le projet . . . 6

Étape 2 : Créer le fichier .env (une seule fois) . . . 6

Étape 3 : Générer la configuration d'Element . . . 7

Étape 4 : Construire et lancer . . . 7

Étape 5 : Approuver le certificat local . . . 7

Étape 6 : Vérifier . . . 7

Étape 7 : Se connecter . . . 8

Étape 8 : Gérer les agents . . . 8

Étape 9 : Suivre, arrêter, relancer, repartir de zéro . . . 8

Correspondance avec le script . . . 8

1. Vue d'ensemble . . . 10

2. Prérequis . . . 11

3. Installation pas à pas . . . 12

3.1 Récupérer le code . . . 12

3.2 Générer la configuration : ./faso init . . . 12

3.3 Construire et lancer : ./faso demarrer . . . 13

3.4 Ce qui se passe au premier démarrage . . . 13

3.5 Certificat local : ./faso certificat . . . 14

3.6 Vérifier . . . 14

4. Configuration détaillée . . . 15

4.1 Synapse (synapse/conf/homeserver.yaml) . . . 15

4.2 Keycloak . . . 16

4.3 Element . . . 16
4.4 Caddy . . . 16
5. Exploitation courante . . . 17
5.1 Commandes . . . 17
5.2 API interne de la console . . . 17
5.3 Sauvegarde . . . 18
5.4 Restauration (sur une installation vide) . . . 18
5.5 Mise à jour des composants . . . 18
6. Passage en production . . . 19
7. Développement . . . 20
8. Dépannage . . . 21

Ce guide s'adresse à la personne qui installe, configure, exploite ou modifie Faso Cikan. Il décrit ce qui se passe à chaque étape, pour qu'on puisse diagnostiquer un problème sans deviner. Les choix d'architecture sont justifiés dans **ARCHITECTURE.md**, les garanties de sécurité dans **NOTE_SECURITE.md**.

PARTIE 0

Les commandes, dans l'ordre

Chaque étape est détaillée plus bas. Ici, seulement ce qu'il faut taper.

1 Préparer Windows (une seule fois, sinon passez à l'étape 2)

Dans **PowerShell en administrateur** :

```
POWERSHELL
wsl --install
```

Redémarrez le PC, puis installez Docker Desktop (https://www.docker.com/products/docker-desktop/). Dans Docker Desktop, activez *Settings* > *Resources* > *WSL integration* > *Ubuntu*.

Créez le fichier `C:\Users\<vous>\.wslconfig` pour donner assez de mémoire à la construction :

```
FICHIER .wslconfig
[wsl2]
memory=6GB
```

Dans PowerShell, appliquez ce réglage :

```
POWERSHELL
wsl --shutdown
```

2 Ouvrir le terminal Ubuntu et vérifier les outils

```
TERMINAL (BASH)
docker version
docker compose version
python3 --version
```

Les trois commandes doivent répondre sans erreur.

3 Copier le projet dans Linux

```
TERMINAL (BASH)
cp -r "/mnt/c/Users/idris/OneDrive/Desktop/projet-arcynex/hackhathon_faso_cikan/faso-cikan" ~/
cd ~/faso-cikan
sed -i 's/\r$//' faso synapse/conf/*.sh postgres/init/*.sh
chmod +x faso
```

4 Générer la configuration

```
TERMINAL (BASH)
./faso init
```

Facultatif, pour voir les valeurs générées :

```
TERMINAL (BASH)
cat .env
```

5 Construire et lancer (environ 10 minutes la première fois)

```
TERMINAL (BASH)
./faso demarrer
```

6 Faire approuver le certificat local

```
TERMINAL (BASH)
./faso certificat
```

Acceptez la fenêtre Windows, puis **fermez complètement le navigateur et rouvrez-le**.

7 Vérifier

```
TERMINAL (BASH)
./faso etat
curl -s https://matrix.faso.localhost/_matrix/client/versions | head -c 200; echo
curl -s -o /dev/null -w '%{http_code}\n' https://matrix.faso.localhost/_synapse/admin/v1/server_version
./faso tests
```

Le premier `curl` affiche du JSON. Le second doit afficher `404`, ce qui prouve que l'API d'administration n'est pas exposée. Les tests doivent tous passer.

8 Se connecter

Ouvrez https://faso.localhost dans le navigateur, puis demandez le code à usage unique :

```
TERMINAL (BASH)
./faso code 318264B
```

Connectez-vous avec le matricule `318264B`, le mot de passe `Cikan-Demo-2026` et le code affiché.

9 Tester la révocation

```
TERMINAL (BASH)
./faso agents
./faso revoquer 402917M
./faso reintegrer 402917M
```

10 Arrêter, relancer, repartir de zéro

```
TERMINAL (BASH)
./faso arreter
./faso demarrer
./faso reinitialiser
```

`./faso arreter` arrête les services en gardant les données. `./faso demarrer` relance, en quelques secondes cette fois. `./faso reinitialiser` efface tout et demande de taper `SUPPRIMER` pour confirmer.

PARTIE 0 BIS

Sans le script : Docker Compose directement

Le script `./faso` n'est qu'un raccourci. Avec Docker Desktop seul, on peut tout faire depuis **PowerShell 7**, sans WSL dans le terminal, sans bash et sans Python sur le PC. Le script fait trois choses que Compose ne fait pas : générer `.env`, générer `element/config/config.json` et calculer les codes de démonstration. On les fait à la main aux étapes 2 et 3.

Prérequis : Docker Desktop lancé, avec au moins 5 Go de mémoire, et les ports 80 et 443 libres.

1 Se placer dans le projet

POWERSHELL

```
cd "C:\Users\idris\OneDrive\Desktop\projet-arcynex\hackhathon_faso_cikan\faso-cikan"
docker compose version
```

2 Créer le fichier `.env` (une seule fois)

Collez ce bloc en entier. Il tire les secrets au hasard avec le générateur cryptographique de .NET :

POWERSHELL

```
function Secret([int]$n = 40) {
    $c = [char[]]'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789'
    -join (1..$n | ForEach-Object { $c[[Security.Cryptography.RandomNumberGenerator]::GetInt32($c.Length)] })
}
$lignes = @(
    "FASO_DOMAINE=faso.localhost"
    "FASO_SERVER_NAME=faso.gov.bf"
    "POSTGRES_PASSWORD=$(Secret)"
    "KEYCLOAK_DB_PASSWORD=$(Secret)"
    "KEYCLOAK_ADMIN=admin-faso"
    "KEYCLOAK_ADMIN_PASSWORD=$(Secret 24)"
    "SYNAPSE_OIDC_SECRET=$(Secret)"
    "SYNAPSE_REGISTRATION_SHARED_SECRET=$(Secret 64)"
    "SYNAPSE_MACAROON_SECRET=$(Secret 64)"
    "SYNAPSE_FORM_SECRET=$(Secret 64)"
    "CONSOLE_OIDC_SECRET=$(Secret)"
    "CONSOLE_SESSION_SECRET=$(Secret 64)"
    "CONSOLE_API_JETON=$(Secret 48)"
)
if (Test-Path .env) { "Le fichier .env existe déjà, on n'y touche pas." }
else { [IO.File]::WriteAllText("$PWD\.env", ($lignes -join "`n") + "`n") }
```

Le fichier est écrit en fins de ligne Unix : avec des fins de ligne Windows, les mots de passe porteraient un caractère invisible à la fin.

3 Générer la configuration d'Element

Il faut refaire cette étape à chaque changement de domaine.

```powershell
$domaine = (Select-String -Path .env -Pattern '^FASO_DOMAINE=(.*)').Matches[0].Groups[1].Value
$serveur = (Select-String -Path .env -Pattern '^FASO_SERVER_NAME=(.*)').Matches[0].Groups[1].Value
$modele  = Get-Content element\config\config.modele.json -Raw
$config  = $modele.Replace('${FASO_DOMAINE}', $domaine).Replace('${FASO_SERVER_NAME}', $serveur)
[IO.File]::WriteAllText("$PWD\element\config\config.json", $config)
```

> Ne sautez pas cette étape. Si `config.json` n'existe pas, Docker crée un **dossier** vide à sa place et le client ne démarre pas. Dans ce cas, supprimez ce dossier, refaites l'étape 3, puis relancez `docker compose up -d element`.

4 Construire et lancer

Comptez environ 10 minutes la première fois.

```powershell
docker compose up -d --build --wait
```

`--wait` rend la main quand tous les services sont `healthy`, comme le fait le script.

5 Approuver le certificat local

```powershell
New-Item -ItemType Directory -Force certificats | Out-Null
docker compose cp caddy:/data/caddy/pki/authorities/local/root.crt certificats\faso-ca.crt
Import-Certificate -FilePath certificats\faso-ca.crt -CertStoreLocation Cert:\CurrentUser\Root
```

Acceptez la fenêtre de confirmation, puis fermez complètement le navigateur et rouvrez-le.

6 Vérifier

```powershell
docker compose ps
curl.exe -s https://matrix.faso.localhost/_matrix/client/versions
curl.exe -s -o NUL -w "%{http_code}`n" https://matrix.faso.localhost/_synapse/admin/v1/server_version
docker compose --profile tests run --rm tests
```

Le premier `curl` affiche du JSON. Le second doit afficher `404`. Les tests doivent tous passer.

7 Se connecter

Ouvrez https://faso.localhost, puis demandez le code à usage unique :

```powershell
docker run --rm -v "${PWD}\demo:/demo:ro" python:3.12-slim python /demo/code.py 318264B
```

Connectez-vous avec le matricule `318264B`, le mot de passe `Cikan-Demo-2026` et le code affiché.

8 Gérer les agents

```powershell
docker compose exec console python -m app agents
docker compose exec console python -m app revoquer 402917M
docker compose exec console python -m app reintegrer 402917M
```

9 Suivre, arrêter, relancer, repartir de zéro

```powershell
docker compose logs -f --tail 100 synapse
docker compose stop
docker compose up -d --wait
docker compose --profile tests down -v
```

Dans l'ordre :

- la première commande suit les journaux d'un service (Ctrl+C pour sortir) ;
- la deuxième arrête les services en gardant les données ;
- la troisième relance ;
- la dernière **efface tout** : conversations, comptes et journaux.

Correspondance avec le script

`./faso …`	Docker direct
`init`	étape 2
`demarrer`	étape 3 puis `docker compose up -d --build --wait`
`arreter`	`docker compose stop`
`etat`	`docker compose ps`
`journaux svc`	`docker compose logs -f --tail 100 svc`
`agents`, `revoquer M`, `reintegrer M`	`docker compose exec console python -m app …`
`code M`	`docker run --rm -v "${PWD}\demo:/demo:ro" python:3.12-slim python /demo/code.py M`
`certificat`	étape 5
`tests`	`docker compose --profile tests run --rm tests`
`tests-navigateur`	`docker compose --profile tests run --rm navigateur`
`reinitialiser`	`docker compose --profile tests down -v`

> Le projet se trouve dans OneDrive. Ça fonctionne, mais la construction est plus lente, et OneDrive peut verrouiller des fichiers pendant qu'il synchronise. En cas d'erreur bizarre, mettez la synchronisation en pause pendant `docker compose up --build`.

PARTIE 1

Vue d'ensemble

Sept conteneurs, définis dans **docker-compose.yml** (projet Compose `faso-cikan`), tous sur un seul réseau Docker `interne` :

Service	Image	Port interne	Rôle
`caddy`	`caddy:2.10-alpine`	**80, 443 publiés**	seul point d'entrée, TLS, en-têtes de sécurité
`element`	`faso-cikan/element-web:1.12.32-faso` (construite)	80	client web Element, patché
`synapse`	`matrixdotorg/synapse:v1.163.2`	8008	serveur Matrix + module `faso_cikan`
`keycloak`	`quay.io/keycloak/keycloak:26.7.5`	8080 (9000 santé)	identité, TOTP, rôles
`postgres`	`postgres:16-alpine`	5432	bases `synapse` et `keycloak`
`console`	`faso-cikan/console` (construite)	8000 interface, 8001 API	révocation, enrôlement, audit
`tests`, `navigateur`	construites, profil `tests`		tests de bout en bout (lancés à la demande)

Routage par Caddy (**caddy/Caddyfile**) :

Hôte	Destination
`${FASO_DOMAINE}/`	page d'accueil statique (`accueil/`)
`${FASO_DOMAINE}/app/`	`element:80`
`matrix.${FASO_DOMAINE}`	`synapse:8008`, seulement `/_matrix/*` et `/_synapse/client/*` ; la fédération et `/_synapse/admin` répondent 404
`auth.${FASO_DOMAINE}`	`keycloak:8080`
`console.${FASO_DOMAINE}`	`console:8000` (le port 8001 n'est jamais routé)

Volumes persistants :

Volume	Contenu	Critique
`faso-cikan_postgres`	bases Synapse et Keycloak	**oui**
`faso-cikan_synapse`	`signing.key`, médias, base SQLite des expirations, config rendue	**oui** (la clé de signature est irremplaçable)
`faso-cikan_console`	`journal.jsonl` (audit, en ajout seul)	oui
`faso-cikan_caddy_data`	autorité de certification locale ou certificats ACME	régénérable
`faso-cikan_caddy_config`	état interne de Caddy	régénérable

PARTIE 2

Prérequis

Matériel

- Construction : environ **5 Go de RAM** pour Docker (la compilation d'Element est lancée avec `--max-old-space-size=4096`) et environ 10 Go de disque pour les images et le cache.
- Exécution : 2 vCPU et 4 Go de RAM suffisent pour une démonstration ou une petite équipe.

Logiciel

- Docker Engine 24 ou plus récent, avec **Compose v2** et **BuildKit** (le Dockerfile d'Element utilise `RUN --mount=type=cache`).
- Bash, Python 3 et `base64` sur l'hôte : le script `./faso` les utilise pour générer `.env`, rendre la configuration d'Element et calculer les codes TOTP de démonstration.
- Sous Windows : WSL 2 et Docker Desktop avec l'intégration WSL activée pour la distribution utilisée. Travaillez dans le système de fichiers Linux (`~/faso-cikan`), pas dans `/mnt/c/...` : les montages Windows sont lents et OneDrive peut modifier les fichiers pendant la construction.

Réseau

- Les ports 80 et 443 de l'hôte doivent être libres.
- La construction a besoin d'Internet : GitHub (clone d'Element), registre npm, Docker Hub et quay.io. L'exécution n'en a pas besoin.
- En local, `*.localhost` se résout tout seul vers `127.0.0.1`. En production, voir la section 8.

PARTIE 3

Installation pas à pas

3.1 Récupérer le code

```
TERMINAL (BASH)
git clone <dépôt> ~/faso-cikan    # ou copier le dossier
cd ~/faso-cikan
chmod +x faso
```

Si le dossier a transité par Windows, vérifiez les fins de ligne (`file faso` ne doit pas afficher `CRLF`). Sinon, lancez `sed -i 's/\r$//' faso synapse/conf/*.sh postgres/init/*.sh`.

3.2 Générer la configuration : `./faso init`

Cette commande crée `.env` (en mode 600) avec des secrets aléatoires. Elle ne fait rien si le fichier existe déjà.

Variable	Usage	Peut-on la changer plus tard ?
`FASO_DOMAINE`	domaine d'accès ; les sous-domaines `matrix.`, `auth.` et `console.` en dérivent	oui, puis `./faso demarrer`
`FASO_SERVER_NAME`	nom du serveur Matrix, inscrit dans chaque identifiant (`@247851k:faso.gov.bf`)	**non** : il est figé dans la base Synapse dès le premier démarrage
`POSTGRES_PASSWORD`	compte `synapse` de PostgreSQL	non sans intervention dans la base
`KEYCLOAK_DB_PASSWORD`	compte `keycloak` de PostgreSQL	non sans intervention dans la base
`KEYCLOAK_ADMIN` / `KEYCLOAK_ADMIN_PASSWORD`	compte d'administration initial de Keycloak	utilisé seulement à la création
`SYNAPSE_OIDC_SECRET`	secret du client OIDC `synapse` dans Keycloak	à changer aussi dans Keycloak
`CONSOLE_OIDC_SECRET`	secret du client OIDC `faso-console`	à changer aussi dans Keycloak
`SYNAPSE_REGISTRATION_SHARED_SECRET`	permet à la console de créer son compte technique dans Synapse	oui
`SYNAPSE_MACAROON_SECRET`, `SYNAPSE_FORM_SECRET`	secrets internes de Synapse	le changer invalide les jetons en cours
`CONSOLE_SESSION_SECRET`	signature des cookies de la console	oui (déconnecte les sessions)
`CONSOLE_API_JETON`	jeton Bearer de l'API interne de la console (port 8001)	oui

Le realm Keycloak n'est importé **qu'une seule fois**, quand il n'existe pas encore. Après le premier démarrage, modifier les secrets OIDC dans `.env` ne suffit pas : il faut les modifier aussi dans l'administration Keycloak (*Clients* > *synapse/faso-console* > *Credentials*).

3.3 Construire et lancer : `./faso demarrer`

Le script enchaîne :

1. `init` si `.env` n'existe pas ;
2. **rendu de la configuration d'Element** : `element/config/config.modele.json` devient `element/config/config.json`, avec les `${VARIABLES}` remplacées par les valeurs de `.env` ;
3. `docker compose up -d --build` ;
4. une attente, au plus 270 s, que tous les services qui ont un *healthcheck* soient `healthy`.

La première fois, la construction de l'image `element` prend l'essentiel du temps. Elle se déroule ainsi (**element/Dockerfile**) :

1. clone d'Element Web au tag `v1.12.32` ;
2. `pnpm install --frozen-lockfile`, avec un cache BuildKit `faso-pnpm` qui évite de tout retélécharger ;
3. copie de `element/src/faso/` dans `apps/web/src/faso/` ;
4. `git apply` des patchs `element/patches/element-web/*.patch`, puis `patch -p1` des patchs `matrix-js-sdk` dans le SDK installé ;
5. `pnpm build` ;
6. copie du résultat dans l'image officielle `vectorim/element-web:v1.12.32`.

Si un patch ne s'applique plus (version d'Element modifiée), la construction s'arrête à l'étape 4 avec le nom du fichier en cause.

3.4 Ce qui se passe au premier démarrage

Les services démarrent dans l'ordre imposé par les `depends_on` :

1. **postgres** initialise le cluster avec la collation `C`, qu'exige Synapse. Le script **postgres/init/10-keycloak.sh** crée ensuite l'utilisateur et la base `keycloak`. Ce script ne s'exécute que si le volume est vide.
2. **keycloak** démarre avec `start --import-realm` et importe **keycloak/realm/fonction-publique.json**, en y remplaçant `${FASO_DOMAINE}` et les secrets OIDC. Le realm contient :
   - les rôles `agent` et `autorite-revocation` ;
   - les clients `synapse` (callback `https://matrix.<domaine>/_synapse/client/oidc/callback`) et `faso-console` (callback `https://console.<domaine>/rappel`) ;
   - le flux d'authentification `navigateur-2fa` (mot de passe **puis** TOTP obligatoire) ;
   - la politique de mot de passe `length(12) and notUsername and passwordHistory(3)` ;
   - TOTP SHA1, 6 chiffres, 30 s, code non réutilisable ;
   - la protection contre la force brute ;
   - aucune inscription, aucune réinitialisation de mot de passe, aucune connexion par e-mail ;
   - les 11 comptes de démonstration et de test.
3. **caddy** crée son autorité de certification locale dans `caddy_data`.
4. **synapse** passe par **synapse/conf/demarrer.sh** :
   - rend `homeserver.yaml` depuis l'environnement (**rendre_config.py**), et s'arrête si une variable manque ;
   - génère `signing.key` si elle est absente ;
   - ajoute l'autorité locale de Caddy aux autorités approuvées (`SSL_CERT_FILE`), pour pouvoir joindre `https://auth.<domaine>` (découverte OIDC) ;
   - charge le module `faso_cikan` depuis `/modules`, monté en lecture seule.
5. **console** attend que Synapse et Keycloak soient `healthy`, puis sert l'interface sur le port 8000 et l'API sur le port 8001.

3.5 Certificat local : `./faso certificat`

Cette commande copie `caddy:/data/caddy/pki/authorities/local/root.crt` vers `certificats/faso-ca.crt`. Sous WSL, elle l'importe aussi dans `Cert:\CurrentUser\Root` par `powershell.exe`. Ailleurs, importez le fichier à la main dans le magasin du système ou du navigateur (Firefox a son propre magasin).

Si le volume `caddy_data` est supprimé, Caddy crée une nouvelle autorité : il faut alors réimporter le certificat.

3.6 Vérifier

TERMINAL (BASH)

```
./faso etat                                # tous les services « healthy »
curl -s https://matrix.faso.localhost/_matrix/client/versions | head -c 200
curl -s -o /dev/null -w '%{http_code}\n' https://matrix.faso.localhost/_synapse/admin/v1/server_version   # 404 attendu
curl -s https://auth.faso.localhost/realms/fonction-publique/.well-known/openid-configuration | head -c 200
./faso tests                               # 26 tests de bout en bout
```

Les tests (**tests/**) s'exécutent dans un conteneur du réseau `interne`. Ils joignent le service par ses noms publics (alias réseau de Caddy) et font confiance à l'autorité locale par le volume `caddy_data`. Ils couvrent quatre thèmes :

1. le socle (fédération, inscription et API d'administration fermées) ;
2. l'identité (matricule non enrôlé, mauvais code, compte sans rôle) ;
3. la classification (message en clair refusé, étiquette invalide refusée, expiration réelle côté serveur) ;
4. la révocation (jeton invalidé, sortie des salons, reconnexion refusée).

La protection contre la force brute de Keycloak peut verrouiller brièvement les comptes de test : laissez une minute entre deux exécutions. `./faso tests-navigateur` rejoue la démonstration dans Chromium et enregistre des captures dans `tests/navigateur/captures/`.

PARTIE 4

Configuration détaillée

4.1 Synapse (synapse/conf/homeserver.yaml)

Réglages de durcissement :

Réglage	Valeur	Effet
`federation_domain_whitelist: []`, `trusted_key_servers: []`		aucune fédération
`enable_registration: false`, `password_config.enabled: false`		connexion uniquement par OIDC
`oidc_providers[0].attribute_requirements`	`faso_roles = agent`	un compte Keycloak sans le rôle est refusé
`localpart_template`	`{{ user.preferred_username }}`	l'identifiant Matrix vient du matricule
`enable_set_displayname: false`		le nom affiché vient de Keycloak
`backchannel_logout_enabled: true`		une fermeture de session Keycloak déconnecte Synapse
`encryption_enabled_by_default_for_room_type: all`		tout salon est chiffré dès sa création
`presence.enabled: false`		pas de « en ligne » ni de « vu à »
`user_ips_max_age: 1d`		adresses IP gardées un jour
`redaction_retention_period: 1m`		contenu rédacté purgé de la base après une minute
`url_preview_enabled: false`		le serveur ne va jamais chercher une URL
`media_retention.local_media_lifetime: 365d`		médias locaux supprimés au bout d'un an
`max_upload_size: 50M`		taille maximale d'une pièce jointe

Module `faso_cikan` :

Paramètre	Défaut	Rôle
`base`	`/data/faso_expirations.sqlite3`	échéances en attente (survivent aux redémarrages)
`intervalle_secondes`	5	période de la boucle d'expiration
`duree_min_secondes` / `duree_max_secondes`	30 / 604800	bornes acceptées pour `duree`
`comptes_techniques`	`@systeme.console:<server_name>`	comptes exemptés des contrôles (la console)

Le module s'accroche à trois points de Synapse :

- `check_event_for_spam` refuse avec une 403 tout `m.room.message` ou `m.sticker` non chiffré, ainsi que toute étiquette `bf.faso.classification` invalide ;
- `on_new_event` enregistre l'échéance des messages classifiés ;
- une boucle de fond rédacte les messages échus au nom de leur expéditeur, ou, s'il a quitté le salon, d'un modérateur local.

4.2 Keycloak

On modifie le realm importé depuis l'administration (`https://auth.<domaine>/admin`, identifiants de `.env`). Pour qu'une modification du JSON soit prise en compte, il faut supprimer le realm ou repartir d'une base vide : l'import ne remplace jamais un realm existant.

Les attributs des agents (ministère, fonction) et les rôles sont stockés sur les utilisateurs du realm. La console lit et écrit ces données par l'API d'administration de Keycloak, sur le réseau interne (`http://keycloak:8080`).

4.3 Element

- `element/config/config.modele.json` est le modèle versionné. `config.json` est **généré** : ne le modifiez pas et ne le versionnez pas (il est dans `.gitignore`).
- `element/branding/` (CSS, police, emblème) est monté en lecture seule dans `/app/faso`. Une modification se voit en rechargeant la page, sans reconstruire l'image.
- Le code ajouté (`element/src/faso/`) et les patchs sont compilés dans l'image. Toute modification demande une reconstruction (voir la section 7).

4.4 Caddy

Les en-têtes appliqués partout sont HSTS, `nosniff`, `Referrer-Policy: no-referrer`, une `Permissions-Policy` restrictive et la suppression de l'en-tête `Server`. Le client est servi avec `X-Frame-Options: SAMEORIGIN`, la console avec `DENY`. L'API d'administration de Caddy est désactivée (`admin off`).

PARTIE 5

Exploitation courante

5.1 Commandes

Commande	Équivalent Compose
`./faso etat`	`docker compose ps`
`./faso journaux [svc]`	`docker compose logs -f --tail 100 [svc]`
`./faso arreter`	`docker compose stop`
`./faso agents` / `revoquer M` / `reintegrer M`	`docker compose exec -T console python -m app …`
`./faso reinitialiser`	`docker compose --profile tests down -v` (**supprime tous les volumes**)

Les journaux Docker sont limités à 3 fichiers de 10 Mo par service.

5.2 API interne de la console

L'API n'écoute que sur `console:8001`, dans le réseau `interne`. Elle s'authentifie par `Authorization: Bearer $CONSOLE_API_JETON`.

Méthode	Chemin	Effet
`GET`	`/agents`	liste des agents (matricule, nom, ministère, état)
`POST`	`/agents/{matricule}/revoquer`	révocation, renvoie un rapport JSON
`POST`	`/agents/{matricule}/reintegrer`	réintégration

Une révocation enchaîne :

1. un relevé des sessions, appareils et salons de l'agent ;
2. la désactivation du compte Keycloak et la fermeture de ses sessions, avec une déconnexion backchannel vers Synapse ;
3. la désactivation du compte Synapse avec `erase: false`, qui supprime ses jetons et appareils et le fait sortir de tous les salons ;
4. une ligne dans `/data/journal.jsonl`.

L'enrôlement se fait par l'interface web de la console (`POST /agents` sur le port 8000, protégé par CSRF et réservé au rôle `autorite-revocation`). Il crée l'utilisateur Keycloak avec un mot de passe provisoire et les actions obligatoires « changer le mot de passe » et « configurer le TOTP ».

Pour consulter le journal d'audit :

TERMINAL (BASH)

```
docker compose exec console cat /data/journal.jsonl
```

5.3 Sauvegarde

Arrêtez Synapse pendant la sauvegarde pour que la base et le volume soient cohérents :

TERMINAL (BASH)

```
mkdir -p sauvegardes && D=$(date +%F)
docker compose stop synapse console
docker compose exec -T postgres pg_dumpall -U synapse | gzip > sauvegardes/postgres-$D.sql.gz
for v in synapse console; do
  docker run --rm -v faso-cikan_$v:/v:ro -v "$PWD/sauvegardes":/s alpine \
    tar czf /s/$v-$D.tar.gz -C /v .
done
cp .env sauvegardes/env-$D
docker compose start synapse console
```

Chiffrez ces archives et stockez-les hors site : elles contiennent la clé de signature du serveur et tous les secrets.

5.4 Restauration (sur une installation vide)

TERMINAL (BASH)

```
docker compose up -d postgres            # volume vide, avec le même .env
gunzip -c sauvegardes/postgres-AAAA-MM-JJ.sql.gz | docker compose exec -T postgres psql -U synapse -d postgres
for v in synapse console; do
  docker volume create faso-cikan_$v
  docker run --rm -v faso-cikan_$v:/v -v "$PWD/sauvegardes":/s alpine \
    sh -c "cd /v && tar xzf /s/$v-AAAA-MM-JJ.tar.gz"
done
./faso demarrer
```

Le script d'initialisation de PostgreSQL crée l'utilisateur `keycloak` au premier démarrage. Si `psql` signale que ce rôle existe déjà, l'erreur est sans conséquence.

5.5 Mise à jour des composants

Les versions sont figées dans `docker-compose.yml` et dans l'argument `ELEMENT_VERSION`. Pour en changer :

1. faites une sauvegarde ;
2. modifiez la version ;
3. pour Element ou le SDK, vérifiez que les patchs s'appliquent toujours ;
4. lancez `./faso demarrer`, puis `./faso tests`.

Synapse migre son schéma tout seul au démarrage. Ne sautez pas plusieurs versions majeures de Keycloak sans lire leurs notes de migration.

PARTIE 6

Passage en production

1. **DNS** : créez quatre enregistrements A ou AAAA vers le serveur : `<domaine>`, `matrix.<domaine>`, `auth.<domaine>` et `console.<domaine>`.
2. **`.env`** : `FASO_DOMAINE=<domaine>`. Choisissez `FASO_SERVER_NAME` **définitivement** avant le premier démarrage.
3. **Certificats** : avec un domaine public joignable sur les ports 80 et 443, Caddy obtient automatiquement des certificats Let's Encrypt. Sur un réseau fermé sans accès ACME, ajoutez une directive `tls <cert> <clé>` par bloc dans le Caddyfile, avec des certificats de l'IGC de l'État.
4. **Pare-feu** : n'ouvrez que les ports 80 et 443. Restreignez `auth.<domaine>/admin` au réseau d'exploitation, par exemple avec un matcher `@admin path /admin*` et `remote_ip` dans le Caddyfile.
5. **Données de démonstration** : supprimez le dossier `demo/` et les comptes `users` du fichier du realm **avant** le premier démarrage, ou supprimez-les dans Keycloak ensuite.
6. **Autorités** : imposez WebAuthn aux comptes qui ont le rôle `autorite-revocation`.
7. **Sauvegardes** : planifiez la procédure de la section 5.3 (cron) et testez une restauration.
8. **Audit** : faites relire les patchs du client et le module serveur par un tiers de confiance.

PARTIE 7

Développement

Modifier…	Puis
`synapse/modules/faso_cikan/*.py`	`docker compose restart synapse` (le module est monté, pas copié)
`synapse/conf/homeserver.yaml`	`docker compose restart synapse` (la configuration est rendue à chaque démarrage)
`console/app/**`	`docker compose up -d --build console`
`element/branding/**`	recharger la page
`element/config/config.modele.json`	`./faso demarrer` (régénère `config.json`)
`element/src/faso/**` ou `element/patches/**`	`docker compose build element && docker compose up -d element`
`accueil/**`, `keycloak/themes/faso/**`	recharger la page (pour le thème Keycloak, vider son cache ou redémarrer `keycloak`)
`caddy/Caddyfile`	`docker compose restart caddy`

Régénérer un patch Element. Partez d'un clone propre au même tag, faites vos modifications, puis :

TERMINAL (BASH)
```
git clone --depth 1 --branch v1.12.32 https://github.com/element-hq/element-web.git /tmp/ew
cd /tmp/ew && git apply ~/faso-cikan/element/patches/element-web/0001-faso-cikan.patch
# … modifications …
git diff > ~/faso-cikan/element/patches/element-web/0001-faso-cikan.patch
```

Gardez les modifications des fichiers d'Element au strict minimum : la logique va dans `element/src/faso/`, les patchs ne servent qu'à la brancher.

Format de l'étiquette (dans le contenu chiffré, et recopiée en clair par le SDK patché) :

JSON
```
"bf.faso.classification": { "niveau": "normal | restreint | confidentiel", "duree": 300 }
```

`duree` est obligatoire pour `confidentiel` et facultative pour `restreint`. Elle est exprimée en secondes, entre les bornes du module.

PARTIE 8

Dépannage

Symptôme	Diagnostic	Solution
`bind: address already in use` sur 80 ou 443	`sudo ss -ltnp 'sport = :443'` (Linux) ou `netstat -ano | findstr :443` (Windows)	arrêter le service en conflit
La construction d'Element est tuée (`exit 137`)	manque de mémoire	au moins 5 Go pour Docker (`.wslconfig` : `memory=6GB`)
`error: patch failed` pendant la construction	version d'Element différente de celle des patchs	revenir à `v1.12.32` ou régénérer les patchs (section 7)
`synapse` reste `unhealthy`, avec `Variables manquantes` dans les journaux	`.env` incomplet	comparer avec la sortie de `./faso init` sur un `.env` neuf
Synapse échoue sur la découverte OIDC (`SSL: CERTIFICATE_VERIFY_FAILED`)	l'autorité de Caddy n'était pas encore créée	`docker compose restart synapse`
Après la connexion Keycloak, « Vous n'êtes pas autorisé »	le compte n'a pas le rôle `agent`	lui ajouter le rôle dans Keycloak
« Invalid redirect uri » sur Keycloak	`FASO_DOMAINE` changé après l'import du realm	corriger les *Valid redirect URIs* des deux clients dans Keycloak
Un message confidentiel ne s'efface pas	`./faso journaux synapse | grep faso_cikan`	vérifier que l'expéditeur ou un modérateur peut rédacter dans le salon
`./faso revoquer` répond `401`	`CONSOLE_API_JETON` changé sans recréer le conteneur	`docker compose up -d console`
Le navigateur refuse le certificat après une réinitialisation	nouvelle autorité Caddy	`./faso certificat`, puis redémarrer le navigateur
`/bin/bash^M: bad interpreter`	fins de ligne CRLF	`sed -i 's/\r$//' faso` (voir la section 3.1)

Pour repartir d'un état vierge : `./faso reinitialiser` (tape `SUPPRIMER`), puis `./faso demarrer`.